

Federated Learning Implementation Guide

Project: Decentralized Plant Leaf Disease Diagnosis System

Architecture: Hybrid VGG-ViT (Vision Transformer)

Framework: Flower (flwr)

1. README: Project Overview

This project implements a federated learning framework to train a decentralized crop disease diagnosis model utilizing an AWS EC2 central server and Kaggle/Local clients. It uses a FedProx strategy to handle non-IID data across different crops and explicit identity tagging to prevent network race conditions.

2. AWS Server Setup (From Scratch)

Step 2.1: Creating an EC2 Instance on AWS

1. Log in to your **AWS Management Console** and navigate to the **EC2 Dashboard**.
2. Click the orange **Launch instance** button.
3. **Name:** Enter `f1-server`.
4. **OS Images (AMI):** Select **Ubuntu** and choose *Ubuntu Server 22.04 LTS*.
5. **Instance Type:** Select `t3.micro` (Free Tier eligible).
6. **Key Pair:** Click *Create new key pair*. Name it `FL-Flower-keypair`, choose **RSA** and **.pem** format. Click **Create key pair** (this will automatically download the `.pem` file to your computer).
7. **Network Settings:** Check **Allow SSH traffic from** and select *My IP*. Click *Edit*, click **Add security group rule**, set Type to **Custom TCP**, Port to **8080**, and Source to **Anywhere (0.0.0.0/0)**.

8. **Configure Storage:** Increase default 8GB to **16GB** to prevent disk space errors with the swap file.
9. Click **Launch instance**. Find your **Public IPv4 address** in the Instances dashboard.

Step 2.2: Local CMD Preparation (Windows)

Move the downloaded `FL-Flower-keypair.pem` to your project folder. Open Command Prompt, navigate to that folder, and run:

```
icaccls "FL-Flower-keypair.pem" /inheritance:r  
icaccls "FL-Flower-keypair.pem" /grant:r "%username%:R"
```

Step 2.3: Connect to AWS

```
ssh -i "FL-Flower-keypair.pem" ubuntu@YOUR_PUBLIC_IP
```

Step 2.4: Environment Configuration

```
sudo apt update && sudo apt upgrade -y  
sudo fallocate -l 4G /swapfile  
sudo chmod 600 /swapfile  
sudo mkswap /swapfile  
sudo swapon /swapfile  
sudo apt install tmux python3-pip -y  
pip install flwr torch torchvision
```

3. Server-Side Execution (AWS)

Step 3.1: fl_server.py

Create the server script (`nano fl_server.py`) and paste:

```
import flwr as fl  
from typing import List, Tuple, Dict  
  
def aggregate_evaluate_metrics(metrics: List[Tuple[int, Dict[str, float]]]) -> Dict[str, float]:
```

```

    accuracies = []
    print("\n--- Round Metrics ---")
    for num_examples, m in metrics:
        client_id = m.get("client_name", "Unknown")
        acc = m["accuracy"]
        accuracies.append(acc)
        print(f"✅ Client: {client_id:10} | Test Accuracy: {acc:.4f}")
    return {"accuracy": sum(accuracies) / len(accuracies) if accuracies
    else 0}

strategy = fl.server.strategy.FedProx(
    proximal_mu=0.01,
    fraction_fit=1.0,
    min_fit_clients=3,
    min_available_clients=3,
    evaluate_metrics_aggregation_fn=aggregate_evaluate_metrics,
)

if __name__ == "__main__":
    fl.server.start_server(
        server_address="0.0.0.0:8080",
        config=fl.server.ServerConfig(num_rounds=40),
        strategy=strategy,
    )

```

Step 3.2: Run in Background

```

tmux new -s fed_server
python3 fl_server.py
# To Detach and leave running: Press Ctrl + B, then D

```

4. Client-Side Execution (Kaggle/Local)

Paste this code into your client notebooks. Update `client_name` at the bottom for each node.

```

class FlowerClient(fl.client.NumPyClient):
    def __init__(self, model, train_loader, test_loader, client_name):
        self.model = model
        self.train_loader = train_loader
        self.test_loader = test_loader
        self.client_name = client_name

```

```

def fit(self, parameters, config):
    self.set_parameters(parameters)
    train(self.model, self.train_loader, epochs=1)
    return self.get_parameters(config={}), len(self.train_loader), {}

def evaluate(self, parameters, config):
    self.set_parameters(parameters)
    loss, accuracy = test(self.model, self.test_loader)
    return float(loss), len(self.test_loader), {
        "accuracy": float(accuracy),
        "client_name": self.client_name
    }

fl.client.start_numpy_client(
    server_address="YOUR_AWS_IP:8080",
    client=FlowerClient(model, train_loader, test_loader, "Apple")
)

```

5. Data Retrieval

Once complete, exit AWS (**exit**) and run this in your local Windows CMD. Don't miss the dot **.** at the end.

```

scp -i "FL-Flower-keypair.pem" ubuntu@YOUR_AWS_IP:~/
federated_results.txt .

```